My Role: Data Analysis Engineer

FranchiseOps AI — Agentic Franchise Management System

This document maps my responsibilities as **Data Analysis Engineer** on the FranchiseOps AI project onto the project's four milestones, and walks through the exact implementation steps, tools, and outputs for the pipeline I own: raw data → cleaning → KPIs → ML prediction → root-cause reasoning. This is the layer that feeds the Outlet Agent, Inventory Agent, and ultimately the Franchise Intelligence Engine.

1. Where This Role Sits in the Architecture

The system is built as specialized agents reporting into a central Franchise Intelligence Engine. My work as Data Analysis Engineer is the data foundation every agent depends on:

- Turning raw operational data (sales, inventory, staff, marketing, audit) into clean, analysis-ready tables
- Calculating exact KPIs and a composite outlet performance score (SQL/Python — not the LLM)
- Building ML models for demand forecasting and underperformance risk
- Producing the structured, cross-domain root-cause output that the LLM turns into a natural-language explanation

2. Responsibilities Mapped to Project Milestones

Milestone	My Responsibility as Data Analysis Engineer
M1 — Outlet Intelligence	Build outlet & sales data pipeline; compute revenue, growth, performance score; detect underperform
M2 — Inventory Intelligence	Clean stock-movement data; compute wastage rate and stock-out risk; build the demand-forecasting
M3 — Workforce + Marketing	Compute attendance/absenteeism and peak-hour staffing KPIs; calculate campaign ROI; feed these
M4 — Full Agentic System	Merge all domain KPIs into one root-cause reasoning layer that identifies priority issues and hands st

3. Step-by-Step Implementation

Step 1: Generate / Ingest Raw Data

What it does: Simulates (or in production, pulls from) POS, ERP, HR, marketing and compliance systems: sales, inventory, staff attendance, marketing campaigns, audit records for every outlet.

Why it matters: Nothing downstream works without a reliable, complete raw data layer covering every agent's domain.

Tools: Python, Pandas, NumPy

```
sales = generate_daily_sales(outlets, days=120)
inventory = generate_stock_movements(outlets, items)
staff, marketing, audit = generate_remaining_tables(outlets)
save_all_to_csv(raw_data_dir)
```

Step 2: Data Cleaning

What it does: Removes duplicates, fixes invalid entries (e.g. negative revenue from refund glitches), and imputes missing values using domain logic — not blind mean-fill.

Why it matters: Bad data produces bad KPIs and bad recommendations; this step is the quality gate for everything after it.

Tools: Pandas

```
sales = sales.drop_duplicates()
sales.loc[sales.revenue < 0, 'revenue'] = np.nan
sales['revenue'] = sales.groupby('outlet_id')['revenue']\
    .transform(lambda s: s.fillna(s.rolling(7, center=True).median()))
# inventory: rebuild closing_stock from the flow equation
inventory['closing_stock'] = inventory['closing_stock'].fillna(
    inventory.opening_stock + inventory.received_qty
    - inventory.consumed_qty - inventory.wastage_qty)
```

Step 3: KPI Engine + Performance Scoring

What it does: Calculates exact KPIs per outlet — revenue growth, wastage rate, absenteeism, compliance score, campaign ROI — then blends them into a single weighted performance_score (0-100) and classifies each outlet as Healthy / At Risk / Critical.

Why it matters: This is the number every dashboard and downstream agent reads. It must be exact math, explainable, and reproducible — not a black box.

Tools: Pandas (SQL-equivalent aggregation logic)

```
score = (0.35*norm(avg_daily_revenue) + 0.15*norm(growth_pct)
  + 0.15*norm(wastage_rate, lower_is_better=True)
  + 0.15*norm(absenteeism, lower_is_better=True)
  + 0.10*norm(compliance_score) + 0.10*norm(campaign_roi))
perf['performance_score'] = round(score * 100, 1)
perf['health_status'] = perf.performance_score.apply(classify)
```

Step 4: ML Layer — Forecasting & Risk Prediction

What it does: Trains an XGBoost regressor for 7-day revenue forecasting per outlet (lag + rolling features), and an XGBoost classifier that ranks which operational factors (absenteeism, understaffing, wastage, compliance) most strongly predict underperformance.

Why it matters: Turns historical data into forward-looking signal — the 'predicts future issues' capability the project calls for — while keeping the LLM out of numeric prediction, per project guidance.

Tools: scikit-learn, XGBoost

```
reg = XGBRegressor(n_estimators=200, max_depth=4)
reg.fit(X_train[lag_and_rolling_features], y_train)
clf = XGBClassifier(n_estimators=100, max_depth=3)
clf.fit(kpi_features, at_risk_label)
importances = clf.feature_importances_  # explains WHY, not just WHAT
```

Step 5: Root-Cause Reasoning Engine

What it does: Scans every outlet's KPIs across all five domains against defined thresholds, ranks issues by severity, and produces a structured JSON object per outlet with findings + a recommendation.

Why it matters: This is the exact-logic layer behind the Franchise Intelligence Engine. It stays deterministic and auditable; the LLM's only job downstream is turning this JSON into a natural-language explanation.

Tools: Python (rule engine), JSON output

```
if wastage_rate > 3.0: issues.append('Inventory Agent: wastage above threshold')
if peak_understaffed_pct > 15.0: issues.append('Staff Agent: peak-hour shortage')
issues_sorted = sorted(issues, key=severity, reverse=True)
recommendation = build_recommendation(issues_sorted)
json.dump(root_cause_results, f, indent=2)
```

4. Deliverables Produced

File	Contents
`raw_data/*.csv`	Outlets, sales, inventory, staff, marketing, audit — raw source tables
`processed/*_clean.csv`	Cleaned, analysis-ready versions of every raw table
`outlet_performance_kpis.csv`	One row per outlet: KPIs, performance_score, health_status
`revenue_forecast_7day.csv`	Per-outlet 7-day forward revenue forecast
`risk_model_features.csv`	Feature set + labels behind the underperformance risk model
`root_cause_analysis.json`	Structured per-outlet diagnosis + recommendation (LLM-ready)

5. Key Design Decisions (for review / demo defense)

- **Imputation is domain-aware, not generic:** missing revenue uses a 7-day rolling median per outlet; missing stock is recomputed from the flow equation rather than statistically guessed.
- **Performance score is a transparent weighted blend** (35% revenue level, 15% growth, 15% wastage, 15% staffing, 10% compliance, 10% marketing ROI) — defensible in front of a franchise owner, unlike a black-box model.
- **The risk classifier is honestly scoped:** with 20 outlets at a single latest-snapshot, it's good for feature-importance-driven explanation, not held-out prediction. Production should train on outlet-week snapshots (thousands of rows).
- **Root-cause detection is rule-based on purpose** — deterministic and auditable, consistent with the project's own guidance to reserve the LLM for reasoning/explanation, not numeric decisions.

6. Tech Stack Used in This Role

Layer	Technology
Data processing	Python, Pandas, NumPy
Machine learning	scikit-learn, XGBoost
Storage (target)	PostgreSQL (CSV used for local development/demo)
Output format	CSV (dashboard-ready) + JSON (LLM-ready)

7. Next Steps to Extend This Role's Work

- Wrap the KPI engine and root-cause logic into FastAPI endpoints (backend/app/agents/outlet_agent.py)
- Schedule the forecasting model as a nightly Celery job so predictions stay current
- Feed root_cause_analysis.json into the LLM prompt for natural-language dashboard explanations

- Repeat the KPI + ML + root-cause pattern for Inventory, Staff, Marketing, and Audit agents (M2–M4), after the Outlet Agent is stable, per the project's own 2-day M1 scoping guidance

---